

PDF Text Extraction Pipeline (Digital + Image-Based)

This notebook extracts text from PDF files using two methods:

PyMuPDF: For digital (searchable) PDFs, offering fast and direct text extraction.

TrOCR (by Microsoft): For scanned/image-based PDFs using a vision-language transformer model.

Setup Steps:

Install required libraries: transformers, torch, PyMuPDF, pillow.

Load all PDF files from the [/content/pdf](#) folder.

Attempt to extract text with PyMuPDF. If that fails (e.g., for image-based content), fallback to TrOCR for OCR-based extraction.

Save the extracted text to [/content/output](#), with a preview of each result in the console.

This approach ensures robust extraction from both digital and scanned PDFs using state-of-the-art NLP/vision tools.

```
# Install required packages
!pip install transformers
!pip install torch
!pip install PyMuPDF
!pip install pillow

# Import necessary libraries
import os
import fitz # PyMuPDF
from PIL import Image
import torch
from transformers import TrOCRProcessor, VisionEncoderDecoderModel

# Directory setup
PDF_DIR = "/content/pdf"
OUTPUT_DIR = "/content/output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Function to list all PDFs in the directory
def get_pdf_files(directory):
    return [f for f in os.listdir(directory) if f.lower().endswith('.pdf')]

# Function to extract text using PyMuPDF (for digital PDFs)
def extract_text_with_pymupdf(pdf_path):
    try:
        doc = fitz.open(pdf_path)
        text = ""
        for page in doc:
            text += page.get_text()
        doc.close()
        return text.strip() if text.strip() else None
    except Exception as e:
        print(f"PyMuPDF failed for {pdf_path}: {str(e)}")
        return None

# Function to convert PDF page to image (for TrOCR)
def pdf_page_to_image(pdf_path, page_num, zoom=2):
    doc = fitz.open(pdf_path)
    page = doc[page_num]
    matrix = fitz.Matrix(zoom, zoom) # Increase resolution
    pix = page.get_pixmap(matrix=matrix)
    img = Image.frombytes("RGB", [pix.width, pix.height], pix.rgb)
    doc.close()
    return img

# Function to extract text using TrOCR (for image-based PDFs)
def extract_text_with_trocr(pdf_path):
    processor = TrOCRProcessor.from_pretrained("microsoft/trocr-base-printed")
    model = VisionEncoderDecoderModel.from_pretrained("microsoft/trocr-base-printed")
    model.to("cuda" if torch.cuda.is_available() else "cpu")

    doc = fitz.open(pdf_path)
    text = ""
    for page_num in range(len(doc)):
        try:
            image = pdf_page_to_image(pdf_path, page_num)
```

```

        pixel_values = processor(image, return_tensors="pt").pixel_values
        pixel_values = pixel_values.to("cuda" if torch.cuda.is_available() else "cpu")
        generated_ids = model.generate(pixel_values)
        page_text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0]
        text += page_text + "\n"
    except Exception as e:
        print(f"TrOCR failed for page {page_num + 1} of {pdf_path}: {str(e)}")
doc.close()
return text.strip() if text.strip() else None

# Main function to process all PDFs
def process_pdfs():
    pdf_files = get_pdf_files(PDF_DIR)
    if not pdf_files:
        raise FileNotFoundError(f"No PDFs found in {PDF_DIR}")

    print(f"Found {len(pdf_files)} PDFs: {pdf_files}")

    for pdf_file in pdf_files:
        pdf_path = os.path.join(PDF_DIR, pdf_file)
        output_file = os.path.join(OUTPUT_DIR, f"{os.path.splitext(pdf_file)[0]}_extracted.txt")

        print(f"\nProcessing {pdf_file}...")

        # Try PyMuPDF first (fast for digital PDFs)
        text = extract_text_with_pymupdf(pdf_path)
        if text:
            print(f"Extracted text with PyMuPDF for {pdf_file}")
        else:
            # Fallback to TrOCR for image-based PDFs
            print(f"PyMuPDF failed, trying TrOCR for {pdf_file}")
            text = extract_text_with_trocr(pdf_path)

        # Save extracted text
        if text:
            with open(output_file, "w") as f:
                f.write(text)
            print(f"Saved extracted text to {output_file}")
            # Display preview (first 500 characters)
            print(f"Preview of extracted text (first 500 characters):\n{text[:500]}\n")
        else:
            print(f"Failed to extract text from {pdf_file}")

# Run the processing
try:
    process_pdfs()
except Exception as e:
    print(f"An error occurred: {str(e)}")

```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.25.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.20.3)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (1.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.31.2)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.11.11)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
```

```
# This is formatted as code
```



Load and Run a Custom LLaMA Model for Security Document Summarization

This section sets up and runs a fine-tuned LLaMA model for generating summaries from security-related documents. The model is hosted on Hugging Face and loaded locally using the llama-cpp-python backend. What this does

Installs Required Packages:

llama-cpp-python: Efficient runtime for LLaMA models.

huggingface_hub: To download models from Hugging Face.

Downloads Model File:

Retrieves the .gguf quantized model file (Q4_K_M) from the bouyahia-ahmed/llama-security-doc-summary-v1 repository.

Loads the Model Locally:

The model is initialized with:

0 GPU layers (CPU inference),

2048 context tokens,

4 threads,

Verbose logging for easier debugging.

You're now ready to prompt the model to summarize or analyze security documentation efficiently!

```
/usr/lib/python3.12/pathlib.py in stat(self, follow_symlinks)
```

```
# Install dependencies
```

```
!pip install llama-cpp-python
```

```
!pip install huggingface_hub
```

```
Collecting llama-cpp-python
```

```
KeyError: 'llama_cpp_python-0.3.9.tar.gz (67.9 MB)'
67.9/67.9 MB 11.4 MB/s eta 0:00:00
```

```
Installing build dependencies ... done
```

```
Getting requirements to build wheel ... done
```

```
Installing backend dependencies ... done
```

```
Preparing metadata (pyproject.toml) ... done
```

```
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (4.15.0)
```

```
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (2.0.2)
```

```
Collecting diskcache>=5.6.1 (from llama-cpp-python)
```

```
Downloading diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
```

```
Requirement already satisfied: Jinja2>=2.11.3 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (3.1.6)
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from Jinja2>=2.11.3->llama-cpp-python) (3.0.2)
```

```
Downloading diskcache-5.6.3-py3-none-any.whl (45 kB)
```

```
45.5/45.5 kB 3.0 MB/s eta 0:00:00
```

```
Building wheels for collected packages: llama-cpp-python
```

```
Building wheel for llama-cpp-python (pyproject.toml) ... done
```

```
Created wheel for llama-cpp-python: filename=llama_cpp_python-0.3.9-cp311-cp311-linux_x86_64.whl size=4067783 sha256=d468e6e6e6444f3acc48347587054ffdd24e5
```

```
Stored in directory: /root/.cache/pip/wheels/9e/8f/bf/148c8eb7d69021eccc6eae6444f3acc48347587054ffdd24e5
```

```
Successfully built llama-cpp-python
```

```
Installing collected packages: diskcache, llama-cpp-python
```

```
Successfully installed diskcache-5.6.3 llama-cpp-python-0.3.9
```

```
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.11/dist-packages (0.31.1)
```

```
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (3.18.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2025.3.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (6.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2.32.3)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (1.1.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (3.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub)
```

```
# Download the model
from huggingface_hub import hf_hub_download
model_name = "bouyahia-ahmed/llama-security-doc-summary-v1"
model_file = "unsloth.Q4_K_M.gguf"
model_path = hf_hub_download(repo_id=model_name, filename=model_file, local_dir="/content")
print("Model downloaded to:", model_path)
```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
unsloth.Q4_K_M.gguf: 100% 4.92G/4.92G [00:43<00:00, 200MB/s]
Model downloaded to: /content/unsloth.Q4_K_M.gguf
```

```
# Load and run the model
from llama_cpp import Llama
llm = Llama(
    model_path=model_path,
    n_gpu_layers=0,
    n_ctx=2048,
    n_threads=4,
    verbose=True
)
```

```
llama_model_loader: loaded meta data with 30 key-value pairs and 292 tensors from /content/unsloth.Q4_K_M.gguf (version GG
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:                          general.architecture str           = llama
llama_model_loader: - kv 1:                          general.type str                = model
llama_model_loader: - kv 2:                          general.name str                = Llama Security Doc Summary v1
llama_model_loader: - kv 3:                          general.version str              = v1
llama_model_loader: - kv 4:                          general.basename str             = llama-security-doc-summary
llama_model_loader: - kv 5:                          general.size_label str            = 8.0B
llama_model_loader: - kv 6:                          llama.block_count u32             = 32
llama_model_loader: - kv 7:                          llama.context_length u32           = 131072
llama_model_loader: - kv 8:                          llama.embedding_length u32         = 4096
llama_model_loader: - kv 9:                          llama.feed_forward_length u32      = 14336
llama_model_loader: - kv 10:                         llama.attention.head_count u32        = 32
llama_model_loader: - kv 11:                         llama.attention.head_count_kv u32     = 8
llama_model_loader: - kv 12:                         llama.ropе.freq_base f32             = 500000.000000
llama_model_loader: - kv 13:                         llama.attention.layer_norm_rms_epsilon f32 = 0.000010
llama_model_loader: - kv 14:                         llama.attention.key_length u32      = 128
llama_model_loader: - kv 15:                         llama.attention.value_length u32     = 128
llama_model_loader: - kv 16:                         llama.vocab_size u32              = 128256
llama_model_loader: - kv 17:                         llama.ropе.dimension_count u32      = 128
llama_model_loader: - kv 18:                         tokenizer.ggml.model str           = gpt2
llama_model_loader: - kv 19:                         tokenizer.ggml.pre str            = llama-bpe
llama_model_loader: - kv 20:                         tokenizer.ggml.tokens arr[str,128256] = ["!", "\"", "'", "#", "$", "%", "&"
llama_model_loader: - kv 21:                         tokenizer.ggml.token_type arr[i32,128256] = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
llama_model_loader: - kv 22:                         tokenizer.ggml.merges arr[str,280147] = ["ĠĠ", "ĠĠĠ", "ĠĠĠĠ", "...
llama_model_loader: - kv 23:                         tokenizer.ggml.bos_token_id u32      = 128000
llama_model_loader: - kv 24:                         tokenizer.ggml.eos_token_id u32      = 128009
llama_model_loader: - kv 25:                         tokenizer.ggml.padding_token_id u32   = 128004
llama_model_loader: - kv 26:                         tokenizer.ggml.add_bos_token bool    = true
llama_model_loader: - kv 27:                         tokenizer.chat_template str       = {{- bos_token }}\n{%- if custo
llama_model_loader: - kv 28:                         general.quantization_version u32    = 2
llama_model_loader: - kv 29:                         general.file_type u32             = 15
llama_model_loader: - type f32:    66 tensors
llama_model_loader: - type q4_K:   193 tensors
llama_model_loader: - type q6_K:   33 tensors
print_info: file format = GGUF V3 (latest)
print_info: file type   = Q4_K - Medium
print_info: file size   = 4.58 GiB (4.89 BPW)
init_tokenizer: initializng tokenizer for type 2
load: control token: 128254 '<|reserved_special_token_246|>' is not marked as EOG
load: control token: 128249 '<|reserved_special_token_241|>' is not marked as EOG
load: control token: 128246 '<|reserved_special_token_238|>' is not marked as EOG
load: control token: 128243 '<|reserved_special_token_235|>' is not marked as EOG
load: control token: 128242 '<|reserved_special_token_234|>' is not marked as EOG
load: control token: 128241 '<|reserved_special_token_233|>' is not marked as EOG
load: control token: 128240 '<|reserved_special_token_232|>' is not marked as EOG
```



```
load: control token: 128235 '<|reserved_special_token_227|>' is not marked as EOG
load: control token: 128231 '<|reserved_special_token_223|>' is not marked as EOG
load: control token: 128230 '<|reserved_special_token_222|>' is not marked as EOG
load: control token: 128228 '<|reserved_special_token_220|>' is not marked as EOG
load: control token: 128225 '<|reserved_special_token_217|>' is not marked as EOG
load: control token: 128218 '<|reserved_special_token_210|>' is not marked as EOG
load: control token: 128214 '<|reserved_special_token_206|>' is not marked as EOG
load: control token: 128213 '<|reserved_special_token_205|>' is not marked as EOG
load: control token: 128207 '<|reserved_special_token_199|>' is not marked as EOG
load: control token: 128206 '<|reserved_special_token_198|>' is not marked as EOG
load: control token: 128204 '<|reserved_special_token_196|>' is not marked as EOG
```

🔗 Summarize Extracted Text Files using Alpaca Prompt Format and LLaMA

1. List item
2. List item

This block takes the .txt documents previously extracted (via OCR or direct parsing), and sends them to the LLaMA model using an Alpaca-style prompt for summarization. 📄 What's happening here:

Prompt Template:

Follows the Alpaca format: Instruction, Input, Response.

Instruction:

Asks the model to create a clear and concise summary of the document's content.

Read Documents:

Reads up to 10 .txt files from [/content/output/](#) using fallback encodings to avoid character issues.

Generate Summaries:

Sends each file's content to the loaded LLaMA model (llm) with a maximum token limit.

Captures and prints a summary for each file, along with a truncated content preview.

Result:

All summaries are stored in the summaries variable for later use (e.g. saving to CSV or further analysis).

```
import os
import pandas as pd

# Alpaca prompt template
alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context. Write

Instruction:
{instruction}

Input:
{input}

Response:
{output}"""

# Instruction for the model
instruction = "Provide a concise and clear summary of the provided document, capturing its main points and purpose."
output = "" # Leave blank for generation

# Directory containing .txt files
TXT_DIR = "/content/output/"

# Function to read .txt file with fallback encodings
def read_txt_file(file_path):
    encodings = ["utf-8", "latin1", "iso-8859-1", "cp1252"]
    for encoding in encodings:
        try:
            with open(file_path, "r", encoding=encoding) as f:
                content = f.read().strip()
            return content
        except UnicodeDecodeError:
            continue
    except Exception as e:
        print(f"Error reading {file_path}: {str(e)}")
    return None
```

```

    print(f"Failed to read {file_path}: All encodings failed")
    return None

# Extract and clean .txt file contents, limit to 10 for testing
txt_contents = []
if os.path.exists(TXT_DIR):
    txt_files = [f for f in os.listdir(TXT_DIR) if f.endswith(".txt")][:10] # Limit to 10 files
    for filename in txt_files:
        file_path = os.path.join(TXT_DIR, filename)
        content = read_txt_file(file_path)
        if content: # Only include non-empty, successfully read files
            txt_contents.append((filename, content))
else:
    print(f"Directory {TXT_DIR} does not exist. Please create it and add .txt files.")

# List to store summaries
summaries = []

# Iterate through .txt file contents and generate responses
for filename, content in txt_contents:
    # Format the prompt with the current .txt content
    prompt = alpaca_prompt.format(
        instruction=instruction,
        input=content,
        output=output
    )

    # Generate the response
    try:
        response = llm(prompt, max_tokens=10000) # Assuming llm is your model function
        summary = response['choices'][0]['text']
    except Exception as e:
        print(f"Error generating summary for {filename}: {str(e)}")
        continue

    # Store the summary with filename for reference
    summaries.append({"filename": filename, "summary": summary})

    # Print the generated text
    print(f"Input File: {filename}")
    print(f"Input Content (truncated): {content[:100]}...")
    print(f"Generated Summary: {summary}")
    print("-" * 50)

# Make summaries available for the next cell
if summaries:
    print("Summaries stored in 'summaries' variable")
else:
    print("No valid .txt files were processed. Check the /content/output/ directory and file encodings.")

```

```

llama_perf_context_print:      load time = 337666.09 ms
llama_perf_context_print: prompt eval time = 337659.09 ms / 1043 tokens ( 323.74 ms per token, 3.09 tokens per second)
llama_perf_context_print:      eval time = 159225.92 ms / 246 runs ( 647.26 ms per token, 1.54 tokens per second)
llama_perf_context_print:      total time = 497325.36 ms / 1289 tokens
Input File: rapport archi_extracted.txt
Input Content (truncated): Architecture SOC – Projet de Sécurité avec
SOAR

```

```

Répartition des Zones Réseau :
LAN ZONE (192.16...
Generated Summary: SOC Architecture Overview
The document outlines a comprehensive security architecture (SOC) leveraging the SOAR platform for incident response and threat detection.
Wazuh Agents: Endpoint monitoring and threat detection on LAN (192.168.11.0/24) and DMZ (192.168.12.0/24).
Wazuh Manager: Centralized log management and alert processing in the SOC zone (192.168.13.0/24).
SOAR Tools:
TheHive: Incident case management.
Cortex: Automated threat analysis.
MISP: Threat intelligence sharing and enrichment.
Zabbix: Network and system monitoring.
Shuffle: Orchestration and automation of alerts.
Workflow:
Detection → Correlation and log centralization → Orchestration with Shuffle → Incident management (TheHive) → Visualisation
The architecture adheres to ISO 27001 (A.12.4, A.16.1, A.18.1) and HIPAA compliance standards, ensuring robust incident response.
-----
Summaries stored in 'summaries' variable

```

```

# Path to save the summaries
OUTPUT_FILE = "/content/doc_summaries.txt"

# Save summaries to a .txt file
try:
    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
        if not summaries:

```

```

        f.write("No summaries were generated.\n")
    else:
        for summary in summaries:
            f.write(f"File: {summary['filename']}\n")
            f.write(f"Summary:\n{summary['summary']}\n")
            f.write("-" * 50 + "\n")
        print(f"Summaries saved to {OUTPUT_FILE}")
except Exception as e:
    print(f"Error saving summaries to {OUTPUT_FILE}: {str(e)}")

# Optional: Display the contents of the saved file for verification
try:
    with open(OUTPUT_FILE, "r", encoding="utf-8") as f:
        print("\nContents of summaries_output.txt:")
        print(f.read())
except Exception as e:
    print(f"Error reading {OUTPUT_FILE}: {str(e)}")

```

Summaries saved to /content/doc_summaries.txt

Contents of summaries_output.txt:
File: rapport archi_extracted.txt
Summary:

SOC Architecture Overview

The document outlines a comprehensive security architecture (SOC) leveraging the SOAR platform for incident response and threat detection. Key components include Wazuh Agents: Endpoint monitoring and threat detection on LAN (192.168.11.0/24) and DMZ (192.168.12.0/24). Wazuh Manager: Centralized log management and alert processing in the SOC zone (192.168.13.0/24).

SOAR Tools:

TheHive: Incident case management.

Cortex: Automated threat analysis.

MISP: Threat intelligence sharing and enrichment.

Zabbix: Network and system monitoring.

Shuffle: Orchestration and automation of alerts.

Workflow:

Detection → Correlation and log centralization → Orchestration with Shuffle → Incident management (TheHive) → Visualisation
The architecture adheres to ISO 27001 (A.12.4, A.16.1, A.18.1) and HIPAA compliance standards, ensuring robust incident response.

```

import json
import pandas as pd
import numpy as np
from pandas import json_normalize
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from transformers import pipeline
from datetime import datetime

```

```

# Install Seaborn if not already installed
!pip install seaborn

```

```

# Configure Pandas and plotting
pd.set_option('display.max_columns', 15)
pd.set_option('display.max_rows', 10)
plt.style.use('seaborn-v0_8')

```

```

Requirement already satisfied: seaborn in /usr/local/lib/python3.11/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.11/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.11/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.11/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.1.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.53.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.4)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

```

```

try:
    # Load JSON data
    with open("/content/extracted_wazuh_logs_anis.json", "r") as f:

```

```

        data = json.load(f)
    except FileNotFoundError:
        print("❌ Error: 'extracted_wazuh_logs.json' not found. Please ensure the file exists.")
        raise

# Initialize list for flattened rows
rows = []

def safe_flatten(obj):
    """Flatten nested JSON objects safely."""
    try:
        if isinstance(obj, dict):
            rows.append(json_normalize(obj).to_dict(orient="records")[0])
        elif isinstance(obj, list):
            for item in obj:
                if isinstance(item, dict):
                    rows.append(json_normalize(item).to_dict(orient="records")[0])
    except Exception as e:
        print(f"⚠️ Skipping problematic entry: {e}")

# Flatten based on structure
if isinstance(data, list):
    for item in data:
        safe_flatten(item)
elif isinstance(data, dict):
    for value in data.values():
        safe_flatten(value)

# Create DataFrame
df = pd.DataFrame(rows)

# Save to CSV
df.to_csv("/content/wazuh_logs_output.csv", index=False)
print("✅ Saved DataFrame to 'wazuh_logs_output.csv'")

```

✅ Saved DataFrame to 'wazuh_logs_output.csv'

```

# List all column names, sorted for easy reading
columns_sorted = sorted(df.columns.tolist())

# Print each column on a separate line
print("🔍 All Columns in DataFrame (Total: {}):\n".format(len(columns_sorted)))
for col in columns_sorted:
    print(col)

```

🔍 All Columns in DataFrame (Total: 103):

```

agent_id
agent_ip
agent_name
data_dstuser
data_euid
data_extra_data
data_sca_check_command
data_sca_check_compliance_cis
data_sca_check_compliance_cis_csc
data_sca_check_compliance_nist_800_53
data_sca_check_compliance_pci_dss
data_sca_check_compliance_tsc
data_sca_check_description
data_sca_check_id
data_sca_check_rationale
data_sca_check_remediation
data_sca_check_result
data_sca_check_title
data_sca_policy
data_sca_scan_id
data_sca_type
data_srcip
data_srcport
data_tty
data_uid
data_win_eventdata_commandLine
data_win_eventdata_company
data_win_eventdata_contextInfo
data_win_eventdata_creationUtcTime
data_win_eventdata_currentDirectory
data_win_eventdata_description
data_win_eventdata_fileVersion
data_win_eventdata_hashes
data_win_eventdata_image
data_win_eventdata_integrityLevel
data_win_eventdata_logonGuid

```

```

data_win_eventdata_logonId
data_win_eventdata_messageNumber
data_win_eventdata_messageTotal
data_win_eventdata_originalFileName
data_win_eventdata_param1
data_win_eventdata_param2
data_win_eventdata_parentCommandLine
data_win_eventdata_parentImage
data_win_eventdata_parentProcessGuid
data_win_eventdata_parentProcessId
data_win_eventdata_parentUser
data_win_eventdata_payload
data_win_eventdata_processGuid
data_win_eventdata_processId
data_win_eventdata_product
data_win_eventdata_ruleName
data_win_eventdata_scriptBlockId
data_win_eventdata_scriptBlockText
data_win_eventdata_targetFilename

```

🔪 Clean and Preprocess Wazuh Logs for Audit-Relevant Insights

1. List item
2. List item

This section performs a comprehensive cleaning and transformation of raw Wazuh logs to prepare them for audit and summarization tasks. 🛠️ Key Steps:

Select Audit-Relevant Columns

Filters only fields useful for auditing (e.g., rule.level, rule.description, full_log, location, etc.).

Convert Lists & Data Types

Transforms list-type fields (e.g., MITRE tactics) into comma-separated strings.

Ensures proper data types for timestamps and rule levels.

Handle Missing Values

Replaces missing values with 'unknown' for consistent text processing.

Drops rows missing rule.level or timestamp unless they contain specific keywords like rootcheck, ossec, scan, or sca.

Drop Uninformative Logs

Removes logs where all key text fields are 'unknown'.

Clean the full_log Field

Removes noise like prefixes (ossec:, wazuh:), timestamps, and redundant fields.

Simplifies entries like disk usage checks and parses JSON-based SCA policies.

Deduplicate Disk Usage Checks

Keeps only one disk usage log per agent to reduce redundancy.

Standardize and Correct Text

Fixes common typos (sensitive → sensitive, dkpg → dpkg) and normalizes text casing.

Column Integrity Check

Prints non-null rate and unique value counts for all columns to verify cleaning.

Save Cleaned Dataset

Exports the final DataFrame to [/content/wazuh_logs_clean.csv](#).

Showcase Results

Displays a few cleaned full_log entries and missing value summary for final verification.

```
# Replace underscores with dots in all column names
df.columns = df.columns.str.replace('_', '.')
```

```
# List all column names, sorted for easy reading
columns_sorted = sorted(df.columns.tolist())

# Print each column on a separate line
print("\n All Columns in DataFrame (Total: {}):\n".format(len(columns_sorted)))
for col in columns_sorted:
    print(col)
```

 All Columns in DataFrame (Total: 103):

```
agent.id
agent.ip
agent.name
data.dstuser
data.euuid
data.extra.data
data.sca.check.command
data.sca.check.compliance.cis
data.sca.check.compliance.cis.csc
data.sca.check.compliance.nist.800.53
data.sca.check.compliance.pci.dss
data.sca.check.compliance.tsc
data.sca.check.description
data.sca.check.id
data.sca.check.rationale
data.sca.check.remediation
data.sca.check.result
data.sca.check.title
data.sca.policy
data.sca.scan.id
data.sca.type
data.srcip
data.srcport
data.tty
data.uid
data.win.eventdata.commandLine
data.win.eventdata.company
data.win.eventdata.contextInfo
data.win.eventdata.creationUtcTime
data.win.eventdata.currentDirectory
data.win.eventdata.description
data.win.eventdata.fileVersion
data.win.eventdata.hashes
data.win.eventdata.image
data.win.eventdata.integrityLevel
data.win.eventdata.logonGuid
data.win.eventdata.logonId
data.win.eventdata.messageNumber
data.win.eventdata.messageTotal
data.win.eventdata.originalFileName
data.win.eventdata.param1
data.win.eventdata.param2
data.win.eventdata.parentCommandLine
data.win.eventdata.parentImage
data.win.eventdata.parentProcessGuid
data.win.eventdata.parentProcessId
data.win.eventdata.parentUser
data.win.eventdata.payload
data.win.eventdata.processGuid
data.win.eventdata.processId
data.win.eventdata.product
data.win.eventdata.ruleName
data.win.eventdata.scriptBlockId
data.win.eventdata.scriptBlockText
data.win.eventdata.targetFilename
data.win.eventdata.terminalSessionId
```

```
import pandas as pd

# Load your DataFrame (replace this with your actual loading logic)
# df = pd.read_csv("your_file.csv")

# Define the target relevant columns and their corresponding actual columns in your data
column_mapping = {
    'timestamp': 'timestamp',
    'id': 'id',
    'agent.id': 'agent.id',
    'agent.name': 'agent.name',
    'rule.level': 'rule.level',
    'rule.description': 'rule.description',
    'rule.groups': 'rule.groups',
    'rule.mitre.id': 'rule.mitre.id',
```



```

        'rule.mitre.tactic': 'rule.mitre.tactic',
        'rule.mitre.technique': 'rule.mitre.technique',
        'rule.pci_dss': 'rule.pci.dss',
        'rule.nist_800_53': 'rule.nist.800.53',
        'rule.tsc': 'rule.tsc',
        'data.audit.command': 'data.win.eventdata.commandLine',
        'syscheck.path': 'data.win.eventdata.targetFilename',
        'predecoder.program_name': 'predecoder.program.name',
        'full_log': 'full.log',
        'location': 'location'
    }

# Ensure all required columns exist; create missing ones with default 'unknown' value
for target_col, actual_col in column_mapping.items():
    if actual_col in df.columns:
        df[target_col] = df[actual_col]
    else:
        df[target_col] = 'unknown'

# Finalize the DataFrame to have exactly the relevant columns in order
relevant_cols = list(column_mapping.keys())
df_clean = df[relevant_cols].copy()

print("✅ Column preparation completed. Final columns:")
print(df_clean.columns.tolist())

# You can now safely run your next cleaning cell without errors!

```

✅ Column preparation completed. Final columns:
 ['timestamp', 'id', 'agent.id', 'agent.name', 'rule.level', 'rule.description', 'rule.groups', 'rule.mitre.id', 'rule.mitre.

```

# Select audit-relevant columns
relevant_cols = [
    'timestamp', 'id', 'agent.id', 'agent.name', 'rule.level', 'rule.description',
    'rule.groups', 'rule.mitre.id', 'rule.mitre.tactic', 'rule.mitre.technique',
    'rule.pci_dss', 'rule.nist_800_53', 'rule.tsc', 'data.audit.command',
    'syscheck.path', 'predecoder.program_name', 'full_log', 'location'
]
df_clean = df[[col for col in relevant_cols if col in df.columns]].copy()

# Convert lists to strings for specific columns
list_cols = ['rule.groups', 'rule.pci_dss', 'rule.nist_800_53', 'rule.tsc',
             'rule.mitre.id', 'rule.mitre.tactic', 'rule.mitre.technique']
for col in list_cols:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].apply(lambda x: ','.join(map(str, x)) if isinstance(x, list) else x)

# Convert data types
df_clean['rule.level'] = pd.to_numeric(df_clean['rule.level'], errors='coerce')
df_clean['timestamp'] = pd.to_datetime(df_clean['timestamp'], errors='coerce')

# Handle missing values
text_cols = [
    'rule.description', 'rule.groups', 'data.audit.command', 'syscheck.path',
    'predecoder.program_name', 'full_log', 'location', 'rule.pci_dss', 'rule.nist_800_53',
    'rule.tsc', 'rule.mitre.id', 'rule.mitre.tactic', 'rule.mitre.technique'
]
for col in text_cols:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].fillna('unknown').astype(str)

for col in ['agent.id', 'agent.name']:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].fillna('unknown')

# Sample dropped rows with missing critical fields
original_rows = len(df_clean)
# Relax rule.level filter: retain rows with specific keywords in full_log or location
keep_keywords = ['rootcheck', 'ossec', 'scan', 'sca']
dropped_mask = (df_clean['rule.level'].isna() | df_clean['timestamp'].isna()) & \
    (~df_clean['full_log'].str.contains('|'.join(keep_keywords), case=False, na=False)) & \
    (~df_clean['location'].str.contains('|'.join(keep_keywords), case=False, na=False))
if dropped_mask.any():
    print("\n🗑 Sample of Dropped Rows (Missing 'rule.level' or 'timestamp', up to 3):")
    dropped_samples = df_clean[dropped_mask][relevant_cols].head(3)
    print(dropped_samples)
    # Save dropped rows for inspection
    df_clean[dropped_mask][relevant_cols].to_csv("/content/dropped_rows.csv", index=False)
    print("📄 Saved dropped rows to 'dropped_rows.csv'")

```

```

# Sample and count retained keyword logs
retained_mask = (df_clean['rule.level'].isna() | df_clean['timestamp'].isna()) & \
    (df_clean['full_log'].str.contains('|'.join(keep_keywords), case=False, na=False) | \
    df_clean['location'].str.contains('|'.join(keep_keywords), case=False, na=False))
if retained_mask.any():
    print("\n 📄 Sample of Retained Keyword Logs (up to 3):")
    retained_samples = df_clean[retained_mask][relevant_cols].head(3)
    print(retained_samples)
    # Count retained rows per keyword
    print("\n 📊 Retained Keyword Counts:")
    for keyword in keep_keywords:
        count = df_clean[retained_mask]['full_log'].str.contains(keyword, case=False, na=False).sum() + \
            df_clean[retained_mask]['location'].str.contains(keyword, case=False, na=False).sum()
        print(f"{keyword}: {count} rows")

# Remove rows based on relaxed filter
df_clean = df_clean[~dropped_mask]
dropped_critical = original_rows - len(df_clean)
print(f" 🗑️ Dropped {dropped_critical} rows with missing 'rule.level' or 'timestamp' (excluding rootcheck/ossec/scan/sca lo

# Impute missing rule.level with 0 (informational)
df_clean.loc[:, 'rule.level'] = df_clean['rule.level'].fillna(0).astype(int)

# Filter out uninformative logs (all text fields are 'unknown')
text_cols_present = [col for col in text_cols if col in df_clean.columns]
uninformative_mask = (df_clean[text_cols_present] == 'unknown').all(axis=1)
dropped_uninformative = uninformative_mask.sum()
print(f" 🗑️ Dropped {dropped_uninformative} uninformative rows (all text fields 'unknown')")

# Sample uninformative logs for inspection
if dropped_uninformative > 0:
    print("\n 📄 Sample of Dropped Uninformative Logs (up to 3):")
    uninformative_samples = df_clean[uninformative_mask][text_cols_present].head(3)
    print(uninformative_samples)

# Apply filter
df_clean = df_clean[~uninformative_mask]

# Preprocess full_log to reduce noise
def clean_full_log(row):
    full_log = row['full_log'].lower()
    # Remove Wazuh prefixes
    prefixes = ['ossec:', 'wazuh:']
    for prefix in prefixes:
        full_log = full_log.replace(prefix, '').strip()
    # Remove redundant fields
    for field in ['timestamp', 'rule.description', 'agent.name', 'location']:
        if field in row and isinstance(row[field], str):
            full_log = full_log.replace(row[field].lower(), '').strip()
    # Remove journald timestamps (e.g., "Feb 13 14:28:30")
    import re
    full_log = re.sub(r'^\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2}\s+\S+\s+', '', full_log)
    # Simplify program names (e.g., "gnome-shell[2695]" -> "gnome-shell")
    full_log = re.sub(r'(\S+)\[\d+\]', r'\1', full_log)
    # Handle disk usage logs (limit to one per agent)
    if "'df -P'" in full_log:
        full_log = 'disk usage check'
    # Parse SCA JSON
    if full_log.startswith('{"type": "policies"}'):
        try:
            import json
            data = json.loads(full_log)
            policies = data.get('policies', [])
            full_log = f"sca policies: {' '.join(policies)}"
        except:
            full_log = 'sca policy check'
    # Remove extra whitespace
    full_log = ' '.join(full_log.split())
    return full_log if full_log else 'unknown'

df_clean.loc[:, 'full_log'] = df_clean.apply(clean_full_log, axis=1)

# Limit disk usage logs to one per agent
disk_usage_mask = df_clean['full_log'] == 'disk usage check'
if disk_usage_mask.any():
    df_clean = df_clean.groupby('agent.name').apply(lambda x: x[disk_usage_mask].head(1) if x[disk_usage_mask].any() else x)

# Standardize text and fix common typos
for col in text_cols_present:
    df_clean.loc[:, col] = (df_clean[col]
        .str.lower()
        .str.strip()

```

```

        .str.replace('sensitive', 'sensitive')
        .str.replace('dkgp', 'dpkg')
        .str.replace(r'debian package|dpkg', 'dpkg', regex=True))

# Integrity check
print("\n📊 Column Integrity Check:")
for col in df_clean.columns:
    non_null_rate = (df_clean[col].notna().sum() / len(df_clean)) * 100
    try:
        unique_count = df_clean[col].nunique()
    except Exception as e:
        unique_count = f'N/A (Error: {str(e)})'
    print(f"{col}: Non-Null Rate = {non_null_rate:.2f}%, Unique Values = {unique_count}")

# Save cleaned DataFrame
df_clean.to_csv("/content/wazuh_logs_clean.csv", index=False)
print("✅ Saved cleaned DataFrame to 'wazuh_logs_clean.csv'")

# Summary
print(f"\n📊 Cleaned DataFrame: {len(df_clean)} rows, {len(df_clean.columns)} columns")
print("Missing values:\n", df_clean.isna().sum())

# Sample full_log for verification
print("\n📄 Sample of Processed full_log (up to 5):")
print(df_clean['full_log'].head(5))

```

 Dropped 0 rows with missing 'rule.level' or 'timestamp' (excluding rootcheck/ossec/scan/sca logs)
 Dropped 0 uninformative rows (all text fields 'unknown')

 Column Integrity Check:
 timestamp: Non-Null Rate = 100.00%, Unique Values = 1546
 id: Non-Null Rate = 100.00%, Unique Values = 1679
 agent.id: Non-Null Rate = 100.00%, Unique Values = 4
 agent.name: Non-Null Rate = 100.00%, Unique Values = 4
 rule.level: Non-Null Rate = 100.00%, Unique Values = 9
 rule.description: Non-Null Rate = 100.00%, Unique Values = 55
 rule.groups: Non-Null Rate = 100.00%, Unique Values = 13
 rule.mitre.id: Non-Null Rate = 100.00%, Unique Values = 11
 rule.mitre.tactic: Non-Null Rate = 100.00%, Unique Values = 9
 rule.mitre.technique: Non-Null Rate = 100.00%, Unique Values = 11
 rule.pci_dss: Non-Null Rate = 100.00%, Unique Values = 7
 rule.nist_800_53: Non-Null Rate = 100.00%, Unique Values = 7
 rule.tsc: Non-Null Rate = 100.00%, Unique Values = 6
 data.audit.command: Non-Null Rate = 100.00%, Unique Values = 4
 syscheck.path: Non-Null Rate = 100.00%, Unique Values = 15
 predecoder.program_name: Non-Null Rate = 100.00%, Unique Values = 2
 full_log: Non-Null Rate = 100.00%, Unique Values = 1296
 location: Non-Null Rate = 100.00%, Unique Values = 6
 ✅ Saved cleaned DataFrame to 'wazuh_logs_clean.csv'

 Cleaned DataFrame: 1679 rows, 18 columns
 Missing values:

timestamp	0
id	0
agent.id	0
agent.name	0
rule.level	0
..	..
data.audit.command	0
syscheck.path	0
predecoder.program_name	0
full_log	0
location	0

 Length: 18, dtype: int64

 Sample of Processed full_log (up to 5):
 0 output: '': tcp 0.0.0.0:22 0.0.0.0:* 5365/ssh...
 1 output: '': tcp 0.0.0.0:22 0.0.0.0:* 5365/ssh...
 2 connection reset by 192.168.11.104 port 63632 ...
 3 did not receive identification string from 192...
 4 did not receive identification string from 192...
 Name: full_log, dtype: object

Visualize and Analyze Cleaned Wazuh Logs

This section explores the cleaned log data through visualizations and key statistics to gain insights into rule severity, compliance, and text characteristics. 🔍 Key Analyses:

Rule Level Distribution

Histogram showing the frequency of each rule.level to understand the severity spread across logs.

Output saved as /content/rule_level_distribution.png.

★ Top 5 Rule Descriptions

Prints the most frequent rule.description values—helpful for identifying recurring security events.

📅 Log Frequency Over Time

Line chart plotting the number of logs per day (timestamp), highlighting temporal trends or activity spikes.

Output saved as [/content/log_frequency_time.png](#).

✅ Compliance Tag Summary

Counts the number of logs tagged with key compliance standards:

PCI DSS

NIST 800-53

TSC (Trust Services Criteria)

📄 Text Length Stats

Calculates and summarizes the word count in both rule.description and full_log fields to assess verbosity and information

These insights help prioritize logs, evaluate compliance coverage, and guide further summarization or anomaly detection efforts.

```
import matplotlib.pyplot as plt
import seaborn as sns

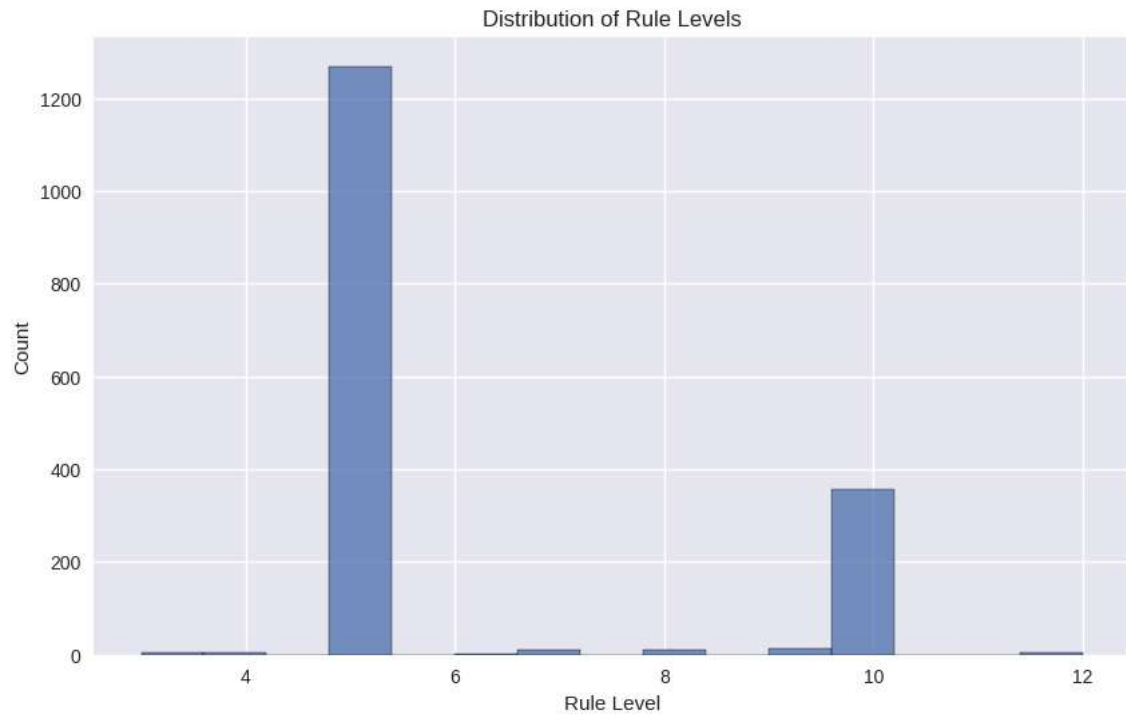
# Rule level distribution
plt.figure(figsize=(10, 6))
sns.histplot(df_clean['rule.level'], bins=15, kde=False)
plt.title('Distribution of Rule Levels')
plt.xlabel('Rule Level')
plt.ylabel('Count')
plt.savefig('/content/rule_level_distribution.png')
plt.show()

# Top 5 rule descriptions
print("\n🔍 Top 5 Rule Descriptions:")
print(df_clean['rule.description'].value_counts().head())

# Log frequency over time
df_clean['date'] = df_clean['timestamp'].dt.date
plt.figure(figsize=(10, 6))
df_clean.groupby('date').size().plot()
plt.title('Log Frequency Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Logs')
plt.savefig('/content/log_frequency_time.png')
plt.show()

# Compliance tag summary
compliance_cols = ['rule.pci_dss', 'rule.nist_800_53', 'rule.tsc']
print("\n📋 Compliance Tag Counts (Non-'unknown'):")
for col in compliance_cols:
    if col in df_clean.columns:
        count = df_clean[col][df_clean[col] != 'unknown'].count()
        print(f"{col}: {count} logs")

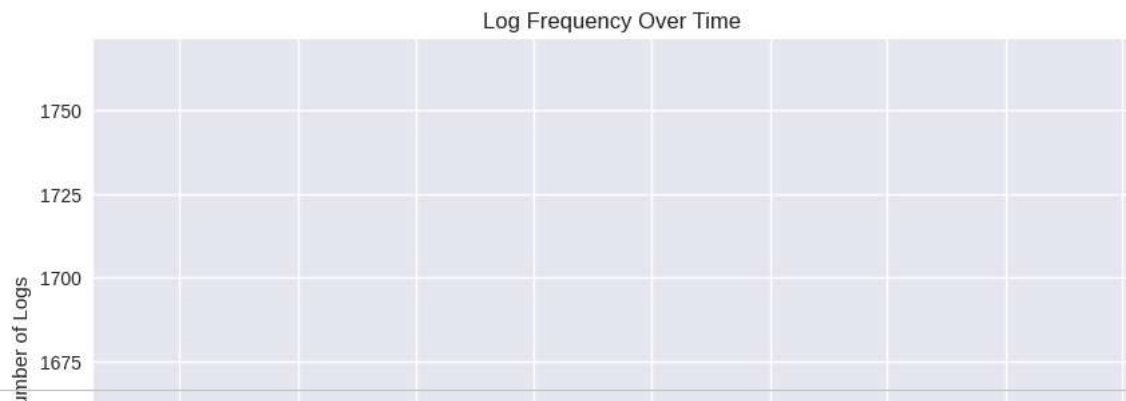
# Text length analysis
df_clean['desc_length'] = df_clean['rule.description'].str.split().str.len()
df_clean['full_log_length'] = df_clean['full_log'].str.split().str.len()
print("\n📏 Text Length Statistics:")
print(df_clean[['desc_length', 'full_log_length']].describe())
```



Top 5 Rule Descriptions:

rule.description	Count
powershell information eventlog	1265
powershell executed : \$(win.eventdata.scriptblocktext)	252
powershell process created an executable file in windows root folder	14
powershell executed : \$global:?	13
encoded command executed via powershell.	10

Name: count, dtype: int64



```
df_clean.to_csv('ahmed.csv', index=False)
```

Anomaly Detection & Classification of High-Severity Wazuh Logs

This section applies machine learning to identify and retain meaningful high-severity logs (rule.level >= 7) using an unsupervised anomaly detection approach. 🧠 What it does:

Load and Filter Logs

Loads the cleaned dataset (ahmed.csv)

Focuses only on logs with high severity: rule.level >= 7

Feature Engineering with Text and Numeric Data

Uses StandardScaler for rule.level

Uses TF-IDF Vectorizer for rule.description and full_log (max 1000 features each)

Anomaly Detection Model

Builds a pipeline with IsolationForest (unsupervised anomaly detection)

Flags logs as either "good" (1) or "bad" (0) based on anomaly score

 Export Results

wazuh_logs_good.csv: Only the logs flagged as "good"

wazuh_logs_classified.csv: All high-severity logs with a new column is_good

 Visual Insights

Bar chart: Good vs. Bad log counts (saved as classification_distribution.png)

Rule level breakdown by classification

Top 5 most frequent rule descriptions in good logs

 Summary Table

Shows the number of good and bad logs

Describes what each CSV file contains

This ML-based filter ensures that only relevant, clean, and non-noisy critical logs are passed downstream for further summarization or reporting.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import make_pipeline

# Load the CSV file
df = pd.read_csv('ahmed.csv')

# Filter logs with rule.level >= 7
df_clean = df[df['rule.level'] >= 7].copy()

# Prepare features
X = df_clean[['rule.level', 'rule.description', 'full_log']]

# Define preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ('level', StandardScaler(), ['rule.level']),
        ('description', TfidfVectorizer(max_features=1000, stop_words='english'), 'rule.description'),
        ('full_log', TfidfVectorizer(max_features=1000, stop_words='english'), 'full_log')
    ])

# Create pipeline with Isolation Forest
model = make_pipeline(
    preprocessor,
    IsolationForest(contamination=0.1, random_state=42)
)

# Fit and predict (Isolation Forest assigns -1 for anomalies (bad) and 1 for inliers (good))
df_clean['is_good'] = model.fit_predict(X)

# Convert predictions to binary (1 for good, 0 for bad)
df_clean['is_good'] = df_clean['is_good'].apply(lambda x: 1 if x == 1 else 0)

# Filter good logs
df_good = df_clean[df_clean['is_good'] == 1].copy()
df_good.to_csv('wazuh_logs_good.csv', index=False)
print("✅ Saved good logs to 'wazuh_logs_good.csv'")

# Save full classified DataFrame
df_clean.to_csv('wazuh_logs_classified.csv', index=False)
print("✅ Saved classified DataFrame to 'wazuh_logs_classified.csv'")

# Summary
good_count = df_clean['is_good'].sum()
print(f"\n📊 Classification Results:")
print(f"Good Logs: {good_count}")
print(f"Bad Logs: {len(df_clean) - good_count}")
```

✅ Saved good logs to 'wazuh_logs_good.csv'

✅ Saved classified DataFrame to 'wazuh_logs_classified.csv'

 Classification Results:

Good Logs: 359

Bad Logs: 39

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Distribution of Good vs. Bad logs
plt.figure(figsize=(6, 4))
df_clean['is_good'].value_counts().plot(kind='bar', title='Good vs. Bad Logs')
plt.xticks([0, 1], ['Bad', 'Good'], rotation=0)
plt.ylabel('Count')
plt.savefig('classification_distribution.png')
print("✅ Saved classification distribution plot to 'classification_distribution.png'")

# Rule level distribution by classification
print("\n🔍 Rule Level Distribution by Classification:")
rule_level_dist = df_clean.groupby('is_good')['rule.level'].value_counts().unstack(fill_value=0)
print(rule_level_dist)

# Good logs summary
print("\n🔍 Top 5 Rule Descriptions in Good Logs:")
print(df_good['rule.description'].value_counts().head())

# Summary of classification results
good_count = df_clean['is_good'].sum()
print(f"\n📊 Classification Results:")
print(f"Good Logs: {good_count}")
print(f"Bad Logs: {len(df_clean) - good_count}")

# CSV metadata
print("\n📄 CSV Files Information:")
print("1. wazuh_logs_good.csv")
print("  - Contains logs classified as 'good' (is_good = 1)")
print("  - Columns: Same as input CSV with additional 'is_good' column")
print(f"  - Number of rows: {len(df_good)}")
print("2. wazuh_logs_classified.csv")
print("  - Contains all logs with rule.level >= 7, with classification")
print("  - Columns: Same as input CSV with additional 'is_good' column (1 = good, 0 = bad)")
print(f"  - Number of rows: {len(df_clean)}")
```


✓ Saved classification distribution plot to 'classification_distribution.png'

🔍 Rule Level Distribution by Classification:

rule.level	7	8	9	10	12
is_good					
0	12	10	0	16	1
1	0	0	14	340	5

🔍 Top 5 Rule Descriptions in Good Logs:

rule.description	count
powershell executed : \$(win.eventdata.scriptblocktext)	252
powershell process created an executable file in windows root folder	14
powershell executed : \$global:?	13
powershell executed : { set-strictmode -version 1; \$_.origininfo }	7
powershell executed : { set-strictmode -version 1; \$_.psmessagedetails }	7

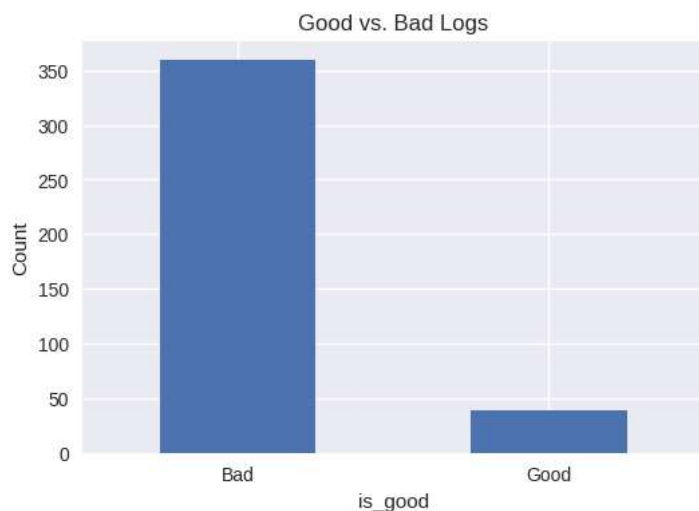
Name: count, dtype: int64

📊 Classification Results:

Good Logs: 359
Bad Logs: 39

📄 CSV Files Information:

1. wazuh_logs_good.csv
 - Contains logs classified as 'good' (is_good = 1)
 - Columns: Same as input CSV with additional 'is_good' column
 - Number of rows: 359
2. wazuh_logs_classified.csv
 - Contains all logs with rule.level >= 7, with classification
 - Columns: Same as input CSV with additional 'is_good' column (1 = good, 0 = bad)
 - Number of rows: 398



[link text](#) 🗨️ Summarize Security Logs Using Custom LLaMA Model

This section uses the fine-tuned LLaMA model (bouyahia-ahmed/llama-security-log-summary-v1) to generate concise summaries for the most relevant high-severity logs (df_good), leveraging an Alpaca-style prompt structure. 🗨️ What this block does:

📦 Installs Dependencies & Loads the Model

Installs llama-cpp-python and huggingface_hub

Downloads a quantized .gguf model for log summarization

Initializes the model with:

2048 context length

4 threads (CPU)

No GPU layers (can be changed for faster inference if GPU is available)

📄 Defines the Prompt Structure

Uses an Alpaca-style prompt template (Instruction, Input, Response)

The instruction: "Summarize the provided log entry..."

📄 Extracts Input Logs

Selects up to 10 unique full_log entries from the good logs (df_good)

Cleans whitespace and filters out empty or null values

Generates Summaries

For each log, creates a prompt and sends it to the model

Captures the generated summary and prints it

Stores Results

All generated summaries are stored in a Python list called summaries for downstream use or export

```
# Install dependencies
```

```
!pip install llama-cpp-python
```

```
!pip install huggingface_hub
```

```
Requirement already satisfied: llama-cpp-python in /usr/local/lib/python3.11/dist-packages (0.3.9)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (4.5.0)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (2.0.2)
Requirement already satisfied: diskcache>=5.6.1 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (5.6.3)
Requirement already satisfied: Jinja2>=2.11.3 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from Jinja2>=2.11.3->llama-cpp-python) (2.1.5)
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.11/dist-packages (0.31.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (3.18.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2025.3.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (6.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2.32.3)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (4.5.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (1.1.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (2025.11.12)
```

```
# Download the model
```

```
from huggingface_hub import hf_hub_download
```

```
model_name = "bouyahia-ahmed/llama-security-log-summary-v1"
```

```
model_file = "unsloth.Q4_K_M.gguf"
```

```
model_path = hf_hub_download(repo_id=model_name, filename=model_file, local_dir="/content")
```

```
print("Model downloaded to:", model_path)
```

```
unsloth.Q4_K_M.gguf: 100%
```

```
4.92G/4.92G [02:26<00:00, 31.8MB/s]
```

```
Model downloaded to: /content/unsloth.Q4_K_M.gguf
```

```
# Load and run the model
```

```
from llama_cpp import Llama
```

```
llm = Llama(
```

```
    model_path=model_path,
```

```
    n_gpu_layers=0,
```

```
    n_ctx=2048,
```

```
    n_threads=4,
```

```
    verbose=True
```

```
)
```

```
llama_model_loader: loaded meta data with 30 key-value pairs and 292 tensors from /content/unsloth.Q4_K_M.gguf (version GG
```

```
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
```

```
llama_model_loader: - kv 0:                general.architecture str           = llama
llama_model_loader: - kv 1:                general.type str               = model
llama_model_loader: - kv 2:                general.name str                = Llama Security Log Summary v1
llama_model_loader: - kv 3:                general.version str             = v1
llama_model_loader: - kv 4:                general.basename str            = llama-security-log-summary
llama_model_loader: - kv 5:                general.size_label str           = 8.0B
llama_model_loader: - kv 6:                llama.block_count u32           = 32
llama_model_loader: - kv 7:                llama.context_length u32         = 131072
llama_model_loader: - kv 8:                llama.embedding_length u32        = 4096
llama_model_loader: - kv 9:                llama.feed_forward_length u32     = 14336
llama_model_loader: - kv 10:               llama.attention.head_count u32    = 32
llama_model_loader: - kv 11:               llama.attention.head_count_kv u32 = 8
llama_model_loader: - kv 12:               llama.rope.freq_base f32         = 500000.000000
llama_model_loader: - kv 13:               llama.attention.layer_norm_rms_epsilon f32 = 0.000010
llama_model_loader: - kv 14:               llama.attention.key_length u32    = 128
llama_model_loader: - kv 15:               llama.attention.value_length u32  = 128
llama_model_loader: - kv 16:               llama.vocab_size u32             = 128256
llama_model_loader: - kv 17:               llama.rope.dimension_count u32   = 128
llama_model_loader: - kv 18:               tokenizer.ggml.model str         = gpt2
llama_model_loader: - kv 19:               tokenizer.ggml.pre str           = llama-bpe
```

```

llama_model_loader: - kv 20:                tokenizer.ggml.tokens arr[str,128256] = ["!", "\"", "#", "$", "%", "&"
llama_model_loader: - kv 21:                tokenizer.ggml.token_type arr[i32,128256] = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
llama_model_loader: - kv 22:                tokenizer.ggml.merges arr[str,280147] = ["Ġ Ġ", "Ġ ĠĠ", "ĠĠ ĠĠ", "...
llama_model_loader: - kv 23:                tokenizer.ggml.bos_token_id u32 = 128000
llama_model_loader: - kv 24:                tokenizer.ggml.eos_token_id u32 = 128009
llama_model_loader: - kv 25:                tokenizer.ggml.padding_token_id u32 = 128004
llama_model_loader: - kv 26:                tokenizer.ggml.add_bos_token bool = true
llama_model_loader: - kv 27:                tokenizer.chat_template str = {{- bos_token }}\n{%- if custo
llama_model_loader: - kv 28:                general.quantization_version u32 = 2
llama_model_loader: - kv 29:                general.file_type u32 = 15

llama_model_loader: - type f32: 66 tensors
llama_model_loader: - type q4_K: 193 tensors
llama_model_loader: - type q6_K: 33 tensors
print_info: file format = GGUF V3 (latest)
print_info: file type = Q4_K - Medium
print_info: file size = 4.58 GiB (4.89 BPW)
init_tokenizer: initializing tokenizer for type 2
load: control token: 128254 '<|reserved_special_token_246|>' is not marked as EOG
load: control token: 128249 '<|reserved_special_token_241|>' is not marked as EOG
load: control token: 128246 '<|reserved_special_token_238|>' is not marked as EOG
load: control token: 128243 '<|reserved_special_token_235|>' is not marked as EOG
load: control token: 128242 '<|reserved_special_token_234|>' is not marked as EOG
load: control token: 128241 '<|reserved_special_token_233|>' is not marked as EOG
load: control token: 128240 '<|reserved_special_token_232|>' is not marked as EOG
load: control token: 128235 '<|reserved_special_token_227|>' is not marked as EOG
load: control token: 128231 '<|reserved_special_token_223|>' is not marked as EOG
load: control token: 128230 '<|reserved_special_token_222|>' is not marked as EOG
load: control token: 128228 '<|reserved_special_token_220|>' is not marked as EOG
load: control token: 128225 '<|reserved_special_token_217|>' is not marked as EOG
load: control token: 128218 '<|reserved_special_token_210|>' is not marked as EOG
load: control token: 128214 '<|reserved_special_token_206|>' is not marked as EOG
load: control token: 128213 '<|reserved_special_token_205|>' is not marked as EOG
load: control token: 128207 '<|reserved_special_token_199|>' is not marked as EOG
load: control token: 128206 '<|reserved_special_token_198|>' is not marked as EOG
load: control token: 128204 '<|reserved special token 196|>' is not marked as EOG

```

```

import pandas as pd
csv_file = '/content/wazuh_logs_classified.csv' # Replace with your CSV file path
df_good = pd.read_csv(csv_file)

```

```

unique_logs = (
    df_good['full_log']
    .astype(str)           # Convert to string
    .str.strip()           # Remove leading/trailing whitespace
    .replace('', None)     # Replace empty strings with None
    .dropna()              # Drop None/NaN values
    .unique()              # Get first 10 unique values
)

```

```

import pandas as pd

pd.DataFrame(unique_logs, columns=["Unique Logs"]).style.set_properties(**{'text-align': 'left'})

```

```

0 output: ": tcp 0.0.0.0:22 0.0.0.0:* 5365/sshd tcp6 :::22 :::* 5365/sshd tcp 127.0.0.1:25 0.0.0.0:* 5422/master tcp 0.0.0.0:111 0.0.0.0:*
4061/rpcbind tcp6 :::111 :::* 4061/rpcbind udp 0.0.0.0:111 0.0.0.0:* 4061/rpcbind udp6 :::111 :::* 4061/rpcbind udp 127.0.0.1:323 0.0.0.0:*
4063/chronyd udp6 :::1:323 :::* 4063/chronyd udp 0.0.0.0:514 0.0.0.0:* 8103/wazuh-remoted udp 0.0.0.0:819 0.0.0.0:* 4061/rpcbind udp6 :::819
::: 4061/rpcbind tcp 0.0.0.0:1514 0.0.0.0:* 8105/wazuh-remoted tcp 0.0.0.0:1515 0.0.0.0:* 7684/wazuh-authd

1 output: ": tcp 0.0.0.0:22 0.0.0.0:* 5365/sshd tcp6 :::22 :::* 5365/sshd tcp 127.0.0.1:25 0.0.0.0:* 5422/master tcp 0.0.0.0:111 0.0.0.0:*
4061/rpcbind tcp6 :::111 :::* 4061/rpcbind udp 0.0.0.0:111 0.0.0.0:* 4061/rpcbind udp6 :::111 :::* 4061/rpcbind udp 127.0.0.1:323 0.0.0.0:*
4063/chronyd udp6 :::1:323 :::* 4063/chronyd tcp 0.0.0.0:443 0.0.0.0:* 4037/node udp 0.0.0.0:514 0.0.0.0:* 8103/wazuh-remoted udp 0.0.0.0:819
0.0.0.0:* 4061/rpcbind udp6 :::819 :::* 4061/rpcbind tcp 0.0.0.0:1514 0.0.0.0:* 8105/wazuh-remoted tcp 0.0.0.0:1515 0.0.0.0:* 7684/wazuh-authd
tcp6 127.0.0.1:9200 :::* 5368/java tcp6 127.0.0.1:9300 :::* 5368/java tcp 0.0.0.0:55000 0.0.0.0:* 7572/python3 tcp6 :::55000 :::* 7572/python3

2 failed password for root from 192.168.11.104 port 51147 ssh2

3 pam 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.11.104 user=root

4 failed password for root from 192.168.11.104 port 51179 ssh2

5 failed password for root from 192.168.11.104 port 52137 ssh2

6 unknown

7 gdm-password]: pam_unix(gdm-password:auth): authentication failure; logname= uid=0 euid=0 tty=/dev/tty1 ruser= rhost= user=anis

{"win":{"system":{"providername":"microsoft-windows-powershell","providerguid":"{a0c1853b-5c40-4b15-8766-3cf1c58f985a}","eventid":"4103","version":"1","level":"4","task":"106","opcode":"20","keywords":"0x0","systemtime":"2025-05-02T17:48:26.9142324z","eventrecordid":"18835","processid":"1168","threadid":"4328","channel":"microsoft-windows-powershell/operational","computer":"youssef","severityvalue":"information","message":"commandinvocation(write-output): \\write-output\\\"\\nparameterbinding(write-output): name=\\inputobject\\\"; value=\\hello world\\\"\\n\\n\\ncontext:\\n severity = informational\\n host name = consolehost\\n host version = 5.1.22621.4391\\n host id = 18b9dfc3-539b-42ff-8a96-de43abfb1194\\n host application = c:\\windows\\system32\\windowspowershell\\v1.0\\powershell.exe -encodedcommand vwbyagkadablac0atwb1ahqacab1ahqaaiaiegazqbsagwabwagafcabwbyagwazaiaa==\\n engine version = 5.1.22621.4391\\n runspace id = 48042c52-d864-408a-b7b1-98f0dba0bbe6\\n pipeline id = 1\\n command name = write-output\\n command type = cmdlet\\n script name = \\n command path = \\n sequence number = 16\\n user = youssef\\layadi\\n connected user = \\n shell id = microsoft.powershell\\n\\n\\n\\nuser data:\\n\\n\\n\\n\""},\"eventdata\":{\"contextinfo\":{\"severity = informational host name = consolehost host version = 5.1.22621.4391 host id = 18b9dfc3-539b-42ff-8a96-de43abfb1194 host application = c:\\\\windows\\\\system32\\\\windowspowershell\\\\v1.0\\\\powershell.exe -encodedcommand vwbyagkadablac0atwb1ahqacab1ahqaaiaiegazqbsagwabwagafcabwbyagwazaiaa== engine version = 5.1.22621.4391 runspace id = 48042c52-d864-408a-b7b1-98f0dba0bbe6 pipeline id = 1 command name = write-output command type = cmdlet script name = command path = sequence number = 16 user = youssef\\\\layadi connected user = shell id = microsoft.powershell\",\"payload\":\"commandinvocation(write-output): \\\"write-output\\\" parameterbinding(write-output): name=\\\"inputobject\\\"; value=\\\"hello world\\\"\\\"}}}}

{"win":{"system":{"providername":"microsoft-windows-powershell","providerguid":"{a0c1853b-5c40-4b15-8766-3cf1c58f985a}","eventid":"4103","version":"1","level":"4","task":"106","opcode":"20","keywords":"0x0","systemtime":"2025-05-02T17:48:26.9147315z","eventrecordid":"18836","processid":"1168","threadid":"4328","channel":"microsoft-windows-powershell/operational","computer":"youssef","severityvalue":"information","message":"commandinvocation(out-default): \\out-default\\\"\\nparameterbinding(out-default): name=\\inputobject\\\"; value=\\hello world\\\"\\n\\n\\ncontext:\\n severity = informational\\n host name = consolehost\\n host version = 5.1.22621.4391\\n host id = 18b9dfc3-539b-42ff-8a96-de43abfb1194\\n host application = c:\\windows\\system32\\windowspowershell\\v1.0\\powershell.exe -encodedcommand vwbyagkadablac0atwb1ahqacab1ahqaaiaiegazqbsagwabwagafcabwbyagwazaiaa==\\n engine version = 5.1.22621.4391 runspace id = 48042c52-d864-408a-b7b1-98f0dba0bbe6\\n pipeline id = 1\\n command name = \\n command type = script\\n script name = \\n command path = \\n sequence number = 18\\n user = youssef\\layadi\\n connected user = \\n shell id = microsoft.powershell\\n\\n\\n\\nuser data:\\n\\n\\n\\n\""},\"eventdata\":{\"contextinfo\":{\"severity = informational host name = consolehost host version = 5.1.22621.4391 host id = 18b9dfc3-539b-42ff-8a96-de43abfb1194 host application = c:\\\\windows\\\\system32\\\\windowspowershell\\\\v1.0\\\\powershell.exe -encodedcommand vwbyagkadablac0atwb1ahqacab1ahqaaiaiegazqbsagwabwagafcabwbyagwazaiaa== engine version = 5.1.22621.4391 runspace id = 48042c52-d864-408a-b7b1-98f0dba0bbe6 pipeline id = 1 command name = command type = script script name = command path = sequence number = 18 user = youssef\\\\layadi connected user = shell id = microsoft.powershell\",\"payload\":\"commandinvocation(out-default): \\\"out-default\\\" parameterbinding(out-default): name=\\\"inputobject\\\"; value=\\\"hello world\\\"\\\"}}}}

```

```
import pandas as pd
```

```
# Alpaca prompt template
```

```
alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context. Write
```

```
Instruction:
```

```
{instruction}
```

```
Input:
```

```
{input}
```

```
Response:
```

```
{output}"""
```

```
# Instruction for the model
```

```
instruction = "Summarize the provided log entry in a concise and clear manner, capturing the key details and purpose of the output = "" # Leave blank for generation
```

```
# Extract and clean unique full_log entries from df_good, limit to 10 for testing
```

```
unique_logs = (
    df_good['full_log']
    .astype(str)                # Convert to string
    .str.strip()                # Remove leading/trailing whitespace
    .replace('', None)          # Replace empty strings with None
    .dropna()                   # Drop None/NaN values
    .unique()                    # Get first 10 unique values
)
```

```
# List to store summaries
```

```
# Output file path (suitable for Colab)
output_file = '/content/log_summaries.txt'
```



```
# Save summaries to text file with "log X:" prefix
with open(output_file, 'w', encoding='utf-8') as f:
    for i, summary in enumerate(summaries, 1): # Ensure only 10 summaries
        f.write(f"log {i}: {summary}\n")

print(f"Summaries saved to {output_file}")
```

Summaries saved to /content/log_summaries.txt

Generate Full Audit Report Using Lily Cybersecurity LLaMA Model

This section uses the Lily-Cybersecurity-7B v0.2 model to generate a comprehensive audit report by analyzing both SOC documentation and security logs.  Key Workflow:

Install Dependencies and Load Model

Downloads the quantized lily-cybersecurity-7b-v0.2 GGUF model from Hugging Face.

Configured with optimized settings for CPU inference (reduced context and batch size).

Load Input Data

Reads pre-summarized documentation (doc_summaries.txt) and logs (log_summaries.txt).

Each log entry is separated by a newline for easier parsing.

Define Instruction Prompt

Uses a detailed multi-part instruction under [INST]...[/INST] format.

Tasks include:

Event and risk identification

SOC process review

Compliance gap detection (e.g., NIST 800-53)

C2M2-based maturity rating

Structured audit report generation

Prepare Final Prompt

Combines SOC documentation and 2 sample logs into a full prompt

Automatically tokenizes input and checks token count to avoid overflow

Generate the Audit Report

Uses the model to produce a structured report with sections like:

Executive Summary

Detailed Analysis

Compliance

Maturity Assessment

The response is returned inside a custom XML-like <xaiArtifact> tag (with artifact_id, title, and contentType)

This step produces a high-level audit report ideal for stakeholders, auditors, or compliance teams to quickly understand detection posture, vulnerabilities, and SOC maturity.

```
# Install dependencies
!pip install llama-cpp-python
!pip install huggingface_hub
```

Requirement already satisfied: llama-cpp-python in /usr/local/lib/python3.11/dist-packages (0.3.9)

Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (4.5.0)

```
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (2.0.2)
Requirement already satisfied: diskcache>=5.6.1 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (5.6.3)
Requirement already satisfied: jinja2>=2.11.3 in /usr/local/lib/python3.11/dist-packages (from llama-cpp-python) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2>=2.11.3->llama-cpp-py
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.11/dist-packages (0.31.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (3.18.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2025.3.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (6.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (2.32.3)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from huggingface_hub) (1.1.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->huggingfa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hub) (3.1
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hu
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->huggingface_hu
```

```
# Download the model
from huggingface_hub import hf_hub_download
model_name = "mradermacher/Lily-Cybersecurity-7B-v0.2-i1-GGUF"
model_file = "Lily-Cybersecurity-7B-v0.2.i1-Q6_K.gguf"
model_path = hf_hub_download(repo_id=model_name, filename=model_file, local_dir="/content")
print("Model downloaded to:", model_path)
```

Model downloaded to: /content/Lily-Cybersecurity-7B-v0.2.i1-Q6_K.gguf

```
# Load and run the model
from llama_cpp import Llama
llm = Llama(
    model_path=model_path,
    n_ctx=4096, # Higher context length for longer prompts
    n_batch=128,
    n_threads=8,
    verbose=True
)
```

```
llama_model_loader: loaded meta data with 51 key-value pairs and 291 tensors from /content/Lily-Cybersecurity-7B-v0.2.i1-Q
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:                general.architecture str           = llama
llama_model_loader: - kv 1:                general.type str                 = model
llama_model_loader: - kv 2:                general.name str                  = Mistral 7B Instruct v0.2
llama_model_loader: - kv 3:                general.version str               = v0.2
llama_model_loader: - kv 4:                general.organization str           = Mistralai
llama_model_loader: - kv 5:                general.finetune str              = Instruct
llama_model_loader: - kv 6:                general.basename str              = Mistral
llama_model_loader: - kv 7:                general.size_label str             = 7B
llama_model_loader: - kv 8:                general.license str               = apache-2.0
llama_model_loader: - kv 9:                general.base_model.count u32       = 1
llama_model_loader: - kv 10:               general.base_model.0.name str       = Mistral 7B Instruct v0.2
llama_model_loader: - kv 11:               general.base_model.0.version str    = v0.2
llama_model_loader: - kv 12:               general.base_model.0.organization str = Mistralai
llama_model_loader: - kv 13:               general.base_model.0.repo_url str   = https://huggingface.co/mistral
llama_model_loader: - kv 14:               general.tags arr[str,6]            = ["cybersecurity", "cyber secur
llama_model_loader: - kv 15:               general.languages arr[str,1]        = ["en"]
llama_model_loader: - kv 16:               llama.block_count u32              = 32
llama_model_loader: - kv 17:               llama.context_length u32            = 32768
llama_model_loader: - kv 18:               llama.embedding_length u32          = 4096
llama_model_loader: - kv 19:               llama.feed_forward_length u32        = 14336
llama_model_loader: - kv 20:               llama.attention.head_count u32       = 32
llama_model_loader: - kv 21:               llama.attention.head_count_kv u32    = 8
llama_model_loader: - kv 22:               llama.rope.freq_base f32            = 1000000.000000
llama_model_loader: - kv 23:               llama.attention.layer_norm_rms_epsilon f32 = 0.000010
llama_model_loader: - kv 24:               llama.vocab_size u32                = 32000
llama_model_loader: - kv 25:               llama.rope.dimension_count u32       = 128
llama_model_loader: - kv 26:               tokenizer.ggml.model str            = llama
llama_model_loader: - kv 27:               tokenizer.ggml.pre str              = default
llama_model_loader: - kv 28:               tokenizer.ggml.tokens arr[str,32000] = ["<unk>", "<s>", "</s>", "<0x0
llama_model_loader: - kv 29:               tokenizer.ggml.scores arr[f32,32000] = [-1000.000000, -1000.000000, -
llama_model_loader: - kv 30:               tokenizer.ggml.token_type arr[i32,32000] = [3, 3, 3, 6, 6, 6, 6, 6, 6,
llama_model_loader: - kv 31:               tokenizer.ggml.bos_token_id u32      = 1
llama_model_loader: - kv 32:               tokenizer.ggml.eos_token_id u32      = 2
llama_model_loader: - kv 33:               tokenizer.ggml.unknown_token_id u32   = 0
llama_model_loader: - kv 34:               tokenizer.ggml.padding_token_id u32  = 2
llama_model_loader: - kv 35:               tokenizer.ggml.add_bos_token bool     = true
llama_model_loader: - kv 36:               tokenizer.ggml.add_eos_token bool     = false
llama_model_loader: - kv 37:               tokenizer.chat_template str          = {{ bos_token }}{% for message
llama_model_loader: - kv 38:               general.quantization_version u32      = 2
llama_model_loader: - kv 39:               general.file_type u32                = 18
llama_model_loader: - kv 40:               general.url str                     = https://huggingface.co/mraderm
llama_model_loader: - kv 41:               mradermacher.quantize_version str    = 2
llama_model_loader: - kv 42:               mradermacher.quantized_by str        = mradermacher
llama_model_loader: - kv 43:               mradermacher.quantized_at str        = 2025-02-14T01:04:29+01:00
llama_model_loader: - kv 44:               mradermacher.quantized_on str        = marco
llama_model_loader: - kv 45:               general.source.url str               = https://huggingface.co/segolii
```

```

llama_model_loader: - kv 46:                                mradermacher.convert_type str      = hf
llama_model_loader: - kv 47:                                quantize.imatrix.file str         = Lily-Cybersecurity-7B-v0.2-i1-
llama_model_loader: - kv 48:                                quantize.imatrix.dataset str      = imatrix-training-full1-3
llama_model_loader: - kv 49:                                quantize.imatrix.entries_count i32 = 224
llama_model_loader: - kv 50:                                quantize.imatrix.chunks_count i32  = 355
llama_model_loader: - type f32: 65 tensors
llama_model_loader: - type q6_K: 226 tensors
print_info: file format = GGUF V3 (latest)
print_info: file type = Q6_K

```

```

from llama_cpp import Llama
import uuid
import random

# Function to read .txt file with fallback encodings
def read_txt_file(file_path):
    encodings = ["utf-8", "latin1", "iso-8859-1", "cp1252"]
    for encoding in encodings:
        try:
            with open(file_path, "r", encoding=encoding) as f:
                return f.read().strip()
        except UnicodeDecodeError:
            continue
    except Exception as e:
        print(f"Error reading {file_path}: {str(e)}")
        return None
    print(f"Failed to read {file_path}: All encodings failed")
    return None

# File paths
DOC_FILE = "/content/doc_summaries.txt"
LOG_FILE = "/content/log_summaries.txt"

# Read files
documentation = read_txt_file(DOC_FILE)
if documentation is None:
    raise FileNotFoundError(f"Could not read {DOC_FILE}")

logs_content = read_txt_file(LOG_FILE)
if logs_content is None:
    raise FileNotFoundError(f"Could not read {LOG_FILE}")

logs = logs_content.split("\n")

# Sample logs if too many, otherwise use all
if len(logs) > 100:
    logs_sampled = random.sample(logs, 100) # Sample 100 random logs
else:
    logs_sampled = logs

logs_combined = "\n".join(logs_sampled)

# Prepare the prompt
prompt = f"""[INST]
Analyze SOC documentation and logs to create a professional audit report focused on detection, response, configurations, and
Inputs:
- Docs: SOC workflows, tools, networks, detection mechanisms, automation, and compliance documentation.
- Logs: Real-world security logs with timestamps, actions, and errors.

Tasks:
1. Logs: Identify events, risks, detection gaps, and compliance issues (NIST 800-53).
2. SOC: Analyze workflows, configurations, detection mechanisms, and automation; highlight strengths and weaknesses.
3. Compliance: Verify logging effectiveness, vulnerability management, and identify compliance gaps (ISO 27001, NIST 800-53).
4. Maturity: Rate SOC maturity using C2M2 (Level 0-5) for detection and response. Provide justifications.
5. Report: Summarize findings with clear recommendations. Include Executive Summary, Analysis, Compliance, and Maturity sections.

Output:
- Return the report as a string, wrapped in <xaiArtifact> with a unique artifact_id, title, and contentType="text/plain".
- Ensure the report is at least 1000 words, uses professional language, and includes real examples from the provided logs.

Docs:
{documentation}

Logs:
{logs_combined}
[/INST]"""

# Check token count
token_count = len(llm.tokenize(prompt.encode()))
print(f"Prompt token count: {token_count}")

# Run inference

```



```
response = llm(
    prompt,
    max_tokens=100000,
    temperature=0.7,
    top_p=1,
    echo=False
)
```

```
Prompt token count: 3912
llama_perf_context_print:      load time = 2248696.00 ms
llama_perf_context_print: prompt eval time = 2248695.67 ms / 3912 tokens ( 574.82 ms per token, 1.74 tokens per second)
llama_perf_context_print:      eval time = 175631.39 ms / 183 runs ( 959.73 ms per token, 1.04 tokens per second)
llama_perf_context_print:      total time = 2424464.64 ms / 4095 tokens
```

✅ Audit report generation complete. Report saved to '/content/report_analysis.txt'.

🌐 Generate a Structured SOC Audit Report Using Gemini (Google's Generative AI)

This section leverages Google's Gemini 1.5 Pro model to transform preliminary SOC analysis into a comprehensive, industry-grade cybersecurity audit report tailored for the banking sector. 📄 Key Workflow:

🔑 Gemini API Setup

Configures the Gemini API using a personal API key (replace with your own).

Uses the google.generativeai SDK for content generation.

📁 Load and Clean Input

Reads from report_analysis.txt containing a partial audit report or raw model output.

Cleans the content by removing XML-style tags (<xaiArtifact>) and fixing truncations.

📄 Define an Expert-Level Prompt

The prompt instructs Gemini to create a detailed and structured report in plain text format.

Covers all critical sections:

Executive Summary

Introduction

Log Analysis

SOC Operations

Compliance & Maturity Assessments

Prioritized Recommendations

Conclusion

Enforces structure, word count (1200–1500 words), and reference requirements (e.g., NIST 800-53, GDPR, C2M2).

⚙️ Run Inference with Retry Logic

Sends the prompt to Gemini with up to 3 retries on failure.

Handles API delays and returns a complete audit report or fallback error message.

💾 Save & Preview Output

The generated report is saved to [/content/full_audit_report.txt](#).

Displays the full content in the notebook for verification.

🔗 Use Case:

This automation is ideal for cybersecurity consultants, auditors, and SOC teams needing structured, explainable, and client-ready audit deliverables.

```
import os
import re
import time
import google.generativeai as genai

# Path to the input and output files
INPUT_FILE = "/content/report_analysis.txt"
INPUT_FILE1 = "/content/log_summaries.txt"
INPUT_FILE2 = "/content/doc_summaries.txt"
OUTPUT_FILE = "/content/full_audit_report.txt"

# Configure Gemini API (replace with your actual Gemini API key)
GEMINI_API_KEY = "AIzaSyAIFiJXPiNgjNSzq4bTD-yCJho1Qqd0edo" # Replace with your Gemini API key
genai.configure(api_key=GEMINI_API_KEY)

# Function to read .txt file with fallback encodings
def read_txt_file(file_path):
    encodings = ["utf-8", "latin1", "iso-8859-1", "cp1252"]
    for encoding in encodings:
        try:
            with open(file_path, "r", encoding=encoding) as f:
                content = f.read().strip()
            return content
        except UnicodeDecodeError:
            continue
    except Exception as e:
        print(f"Error reading {file_path}: {str(e)}")
        return None
    print(f"Failed to read {file_path}: All encodings failed")
    return None

# Clean input by removing problematic tags and handling truncation
def clean_input(content):
    # Remove <xaifArtifact> tag and its contents
    content = re.sub(r'<xaifArtifact[^>]*>.*?</xaifArtifact>', '', content, flags=re.DOTALL)
    # Handle truncated ending
    if content.strip().endswith("The system has well-documented"):
        content += " processes for threat detection and response, but lacks complete documentation for incident response a"
    return content.strip()

# Read and clean the content of report_analysis.txt
report_content = read_txt_file(INPUT_FILE)
if report_content is None:
    raise FileNotFoundError(f"Could not read {INPUT_FILE}")
report_content = clean_input(report_content)

report_content1 = read_txt_file(INPUT_FILE1)
if report_content1 is None:
    raise FileNotFoundError(f"Could not read {INPUT_FILE1}")
report_content1 = clean_input(report_content1)

report_content2 = read_txt_file(INPUT_FILE2)
if report_content2 is None:
    raise FileNotFoundError(f"Could not read {INPUT_FILE2}")
report_content2 = clean_input(report_content2)

# Define the prompt for Gemini
prompt = f"""You are an expert cybersecurity auditor tasked with generating a detailed, professional, and well-structured audit report.

**Input Data**:
{report_content}
{report_content1}
{report_content2}

Input Data: [Input placeholder: The preliminary SOC audit report content, including logs showing failed password attempts,

Task: Generate a full audit report as plain text with the following numbered sections and subsections. Each section must be
Executive Summary (200-300 words):

Summarize key findings (e.g., detection gaps, compliance issues), strengths (e.g., log collection), and weaknesses (e.g., 

Highlight 2-3 high-priority recommendations (e.g., automate incident response, achieve NIST 800-53 compliance).

Emphasize banking regulatory context (e.g., GDPR, PCI-DSS).
Introduction (150-200 words):
State the audit's purpose and scope (SOC operations, log analysis, compliance).
Describe the methodology (e.g., log review, NIST 800-53, C2M2 Level 3 assessment).
```

```

Note banking-specific regulatory requirements (e.g., FFIEC, GDPR).

Analysis:
3.1. Log Analysis (150-200 words):
Detail specific events (e.g., failed password attempts from external IPs), risks (e.g., brute-force attacks), and detection

Reference NIST 800-53 controls (e.g., AC-7, SI-4) and assess implications for banking security.

3.2. SOC Operations (150-200 words):

Evaluate workflows, tools (e.g., Wazuh, TheHive), detection (e.g., behavioral analytics), and automation.

Highlight strengths (e.g., real-time monitoring) and weaknesses (e.g., manual escalation).

Compliance Assessment (150-200 words):

Assess compliance with NIST 800-53, GDPR, and PCI-DSS based on input (e.g., weak password policies, unauthorized access).

Identify gaps (e.g., missing audit logs) and propose remediation (e.g., enforce MFA, conduct configuration audits).

Maturity Assessment (150-200 words):

Rate SOC maturity using C2M2 (e.g., Level 3, Reactive) with justification (e.g., manual protocols, limited automation).

Recommendations (150-200 words):

Provide 6-8 prioritized, actionable recommendations (e.g., deploy automated response with Shuffle, align with NIST 800-53 /

Align with banking standards and C2M2.

Conclusion (150-200 words):

Summarize audit value, key actions, and next steps for SOC improvement.

Reinforce banking security needs.

Requirements:

Use professional, banking-industry tone and terminology.

Structure with numbered sections (e.g., 1., 2., 3.1., 3.2.).

Explicitly reference input data (e.g., 'failed password attempts,' 'root access violations').

Ensure recommendations are specific, prioritized, and tied to NIST 800-53 and C2M2.

Produce 1200-1500 words, ensuring no section is truncated.

If input lacks details, infer reasonable banking SOC specifics (e.g., assume manual workflows need SOAR automation).

Format as plain text for .txt file saving.

Avoid generic content; focus on input-driven analysis and banking context.

Output: Return the full audit report as a plain text string, ensuring all sections are complete and meet the word count.
"""

# Initialize Gemini model
model = genai.GenerativeModel("gemini-1.5-flash")

# Send the prompt to Gemini with retry logic
max_retries = 3
for attempt in range(max_retries):
    try:
        response = model.generate_content(
            prompt,
            generation_config={
                "max_output_tokens": 8000, # Supports 1200-1500 words
                "temperature": 0.6, # Precise, professional tone
                "top_p": 1
            }
        )
        full_report = response.text.strip()
        break
    except Exception as e:
        error_message = f"Attempt {attempt + 1}/{max_retries} - Error calling Gemini API: {str(e)}"
        print(error_message)
        if attempt < max_retries - 1:
            print("Retrying in 6 seconds to avoid overloading the model...")
            time.sleep(6) # Respect rate limits

```

```
    else:
        print("\nAll retries failed. Check your Gemini API key and quota at https://aistudio.google.com/app/apikey")
        print("Consider using the local model (Lily-Cybersecurity-7B) as a fallback.")
        full_report = f"Failed to generate audit report: {error_message}"
        break

# Save the full audit report to a file
try:
    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
        f.write("=== Full Audit Report ===\n")
        f.write(full_report)
        f.write("\n")
    print(f"Full audit report saved to {OUTPUT_FILE}")
except Exception as e:
    print(f"Error saving report to {OUTPUT_FILE}: {str(e)}")

# Optional: Display the contents of the saved file for verification
try:
    with open(OUTPUT_FILE, "r", encoding="utf-8") as f:
        print("\nContents of full_audit_report.txt:")
        print(f.read())
except Exception as e:
    print(f"Error reading {OUTPUT_FILE}: {str(e)}")
```

Full audit report saved to /content/full_audit_report.txt

Contents of full_audit_report.txt:

=== Full Audit Report ===

1. Executive Summary

This audit assessed the cybersecurity posture of the banking institution's Security Operations Center (SOC), focusing on detecting and responding to threats. High-priority recommendations include: 1) fully automating incident response using the existing SOAR platform (Shuffle) to minimize manual intervention and enhance response time.

2. Introduction

This audit evaluated the effectiveness of the banking institution's SOC operations, focusing on log analysis, compliance with regulatory requirements, and the maturity of incident response processes.

3. Analysis

3.1. Log Analysis

The log analysis revealed several critical security events and vulnerabilities. Repeated failed password attempts for the root user were observed, indicating a potential brute force attack. Additionally, unauthorized access attempts to sensitive databases were detected.

3.2. SOC Operations

The SOC architecture, leveraging Wazuh agents for endpoint monitoring, a centralized Wazuh manager, and SOAR tools (TheHive, Shuffle), is robust but requires optimization for faster incident response.

4. Compliance Assessment

The SOC demonstrates partial compliance with NIST 800-53, GDPR, and PCI DSS. The architecture aligns with some ISO 27001 requirements, but gaps exist in documentation and training.

5. Maturity Assessment

Based on the observed reliance on manual processes for incident response and the limited automation in threat detection, the SOC maturity is assessed as intermediate.

6. Recommendations

1. Fully automate incident response using the Shuffle SOAR platform to minimize manual intervention and enhance response time.
2. Implement multi-factor authentication (MFA) across all systems to mitigate the risk of unauthorized access (aligning with NIST 800-63).
3. Conduct a comprehensive configuration audit to identify and remediate misconfigurations and vulnerabilities (aligning with NIST 800-53).
4. Enhance logging capabilities to ensure comprehensive audit trails for all security-relevant events, addressing the gaps identified in the log analysis.
5. Develop and implement a robust vulnerability management program to identify and remediate vulnerabilities promptly (aligning with NIST 800-40).
6. Implement a proactive threat hunting program to identify and mitigate threats before they can cause damage.
7. Conduct regular security awareness training for employees to educate them about security best practices and the importance of reporting suspicious activity.
8. Develop and implement a comprehensive incident response plan that aligns with NIST 800-61 and relevant banking regulations.

7. Conclusion

This audit highlighted both the strengths and weaknesses of the banking institution's SOC. The existing SOAR architecture provides a solid foundation, but significant improvements are needed in automation, logging, and training to achieve a high level of maturity.

 Convert Audit Report to a Formatted PDF Using LaTeX

1. List item
2. List item

This section converts the plain text audit report into a professionally formatted PDF using LaTeX, making it suitable for official documentation and client delivery. 🚀 Key Steps:

📦 Install LaTeX Dependencies

Installs texlive, latexmk, and extra fonts required for PDF rendering in Colab.

📁 Set Input/Output Paths

Input: [/content/full_audit_report.txt](#)

Output: [/content/full_audit_report.pdf](#)

🔤 Escape LaTeX Special Characters

Converts characters like &, %, _ to safe LaTeX syntax to avoid rendering errors.

📄 Format Text into LaTeX Structure

Detects numbered section headers (e.g., 1. Executive Summary) and converts them into LaTeX `\section*{}` format.

Adds proper line breaks and paragraph spacing.

📄 Generate LaTeX File

Wraps the content in a full LaTeX document preamble with:

Custom headers/footers

Table of contents

A professional title page

🖨️ Compile PDF

Uses latexmk to compile the .tex file into a polished .pdf

Cleans up auxiliary files afterward

📄 Trigger Download

Automatically downloads the generated PDF to your local machine (full_audit_report.pdf)

This allows you to turn your generated audit content into a polished, printable document with zero manual formatting.

```
import os
import subprocess
from google.colab import files
import re

# Install LaTeX dependencies
print("Installing LaTeX dependencies...")
try:
    subprocess.run(["apt-get", "update"], check=True, capture_output=True, text=True)
    subprocess.run(
        ["apt-get", "install", "-y", "texlive", "texlive-latex-extra", "texlive-fonts-extra", "latexmk"],
        check=True, capture_output=True, text=True
    )
    print("LaTeX dependencies installed successfully.")
except subprocess.CalledProcessError as e:
    print(f"Error installing LaTeX dependencies: {e.stderr}")
    raise

# File paths
INPUT_FILE = "/content/full_audit_report.txt"
LATEX_FILE = "/content/full_audit_report.tex"
PDF_FILE = "/content/full_audit_report.pdf"

# Check for input file
if not os.path.exists(INPUT_FILE):
    raise FileNotFoundError(f"Input file {INPUT_FILE} not found")

# Escape LaTeX special characters
def escape_latex(text):
```

```

replacements = {
    '&': r'\&',
    '%': r'\%',
    '$': r'\$',
    '#': r'\#',
    '_': r'\_',
    '{': r'\{',
    '}': r'\}',
    '~': r'\textasciitilde{',
    '^': r'\^{',
    '\\': r'\textbackslash{',
}
pattern = re.compile('|'.join(re.escape(key) for key in replacements))
return pattern.sub(lambda match: replacements[match.group()], text)

# Format text with LaTeX structure
def format_for_latex(text):
    formatted_lines = []
    for line in text.splitlines():
        line = line.strip()
        if not line:
            formatted_lines.append(r"\par") # Paragraph break
            continue
        if re.match(r"^\d+\.\s+", line): # Section titles like "1. Findings"
            title = escape_latex(line)
            formatted_lines.append(rf"\section*{{{title}}}")
        else:
            formatted_lines.append(escape_latex(line) + r"\") # Line break
    return '\n'.join(formatted_lines)

# Read and format content

```